

Kaspar-HFT

Open Source

A turn-key CME futures trading system and order book simulator.

VENUE
CME Globex

MARKET DATA
MDP3 SBE

ORDER ENTRY
iLink 3

BOOK
MBP & MBO L3

LICENCE
MIT, open source

From production to simulation. It is a production trading system — multicast market data in, full order books reconstructed order by order, an execution algorithm on top, an order-management session out to the exchange. It is also an order book simulator: the same books rebuilt from recorded packet captures.

The same strategy code, the same execution algorithm and the same book run in PCAP replay, in live paper trading, and against the live exchange. Moving between them is a configuration change, so a backtest exercises the code path that will trade.

WHAT IS IN THE BOX

Feed handler, book, execution logic and exchange session layer all included.

SIMULATION

Three modes off one binary: **PCAP replay**, live multicast paper trading, live iLink execution. Deterministic — a fixed session replays to identical fills.

CONCURRENCY

A custom **C++20 actor framework** — a formal concurrency model, and the same code whether the hot path runs across threads or entirely on one. Scale out by giving actors their own threads; collapse to a single-threaded hot path by grouping them, from the same source either way.

STRATEGY AUTHORING IN C++ OR RUST

C++ in-process actors for the lowest latency, or Rust through an in-process FFI. Either way the strategy runs in the same process as the book.

EXECUTION

Shadow-POV, a passive/aggressive percentage-of-volume algorithm that places orders by following other participants' orders in the market.

MARKET DATA

Full **MDP3** SBE decoder: incremental book updates, trades, order-by-order MBO, instrument definitions, snapshot recovery. Sequence gaps are detected and repaired automatically.

ORDER BOOK

Order-by-order reconstruction, holding every resting order individually. That is what makes queue position, and therefore fill inference, exact.

ORDER ENTRY

iLink 3 session handler: SBE encoding, HMAC authentication, sequence management, negotiation and establishment, primary/secondary failover.

PERFORMANCE

AMD EPYC 9374F · RHEL 9 · g++ 15 · -O3 -march=native

The usual objection to actors is messaging overhead. The answer is `fast_send`, which runs the receiver's handler inline on the caller's thread and returns the reply as a value.

PATH	P50	WHAT IT COSTS
<code>fast_send</code> — inline, same thread	~10 ns	≈ one polymorphic virtual call
Queued send, one Group thread	~90 ns	queue push/pop + dispatch
Queued send, separate threads	~3,370 ns	cross-core mailbox wakeup, twice
Pooled round trip (2 messages)	~92 ns	vs ~128 ns with plain <code>new</code>
Allocator tail (max observed)	~33 μs	vs ~5.5 ms unpooled — the durable win

MEASURED Round trip through the framework — send, handle, reply — one message in flight, from the microbenchmarks in `actors/cpp/perf`.